How-To Library

Informatica

Configuring Pushdown Optimization for PowerExchange for Amazon Redshift Task Using an ODBC Connection

© Copyright Informatica LLC 2019, 2021. © Copyright Informatica LLC 2019. Informatica, the Informatica logo, PowerExchange, and PowerCenter are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/ trademarks.html.

Abstract

You can use pushdown optimization to push transformation logic to source databases or target databases. Use pushdown optimization to improve task performance by using the database resources. When you run a task configured for pushdown optimization, the task converts the transformation logic to an SQL query. The task sends the query to the database and the database executes the query.

This article explains how to set pushdown optimization for the ODBC connection type that uses Amazon ODBC Redshift drivers for mappings in the native environment. This article also explains the supported pushdown optimization functions, operations, and rules and guidelines to use pushdown optimization.

Supported Versions

- PowerExchange for Amazon Redshift 10.2.2

Table of Contents

Overview. 2
Installing the Amazon Redshift ODBC drivers. 3
Configuring System DSN. 3
Configuring a System DSN on Windows. 3
Configuring a System DSN on Linux. 5
Creating an Amazon Redshift ODBC Connection. 6
Importing the Amazon Redshift Data Objects. 7
Creating a Mapping. 7
Supported Pushdown Optimization Functions and Operators. 8
Rules and Guidelines for Functions in Pushdown Optimization. 10

Overview

To optimize the mapping performance in the native environment, you can push transformation logic to the source or target database. You can use full or source pushdown optimization for the ODBC connection type that uses Amazon ODBC Redshift drivers for mappings. Using the ODBC connection type with pushdown optimization enhances the mapping performance.

To read data from and write data to a Amazon Redshift object using the OBDC connection, perform the following steps:

1. Download and install the Amazon Redshift ODBC driver.
2. Configure a system DSN.
3. Create an ODBC connection to access the Amazon Redshift read and write data objects.
4. Import the Amazon Redshift read and write data objects.
5. Create and run a mapping.

Installing the Amazon Redshift ODBC drivers

Download and install the Amazon Redshift ODBC drivers from the AWS website to connect to Amazon Redshift using an ODBC connection.

PowerExchange for Amazon Redshift supports Amazon ODBC Redshift drivers on Windows and Linux systems. Based on the operating system, download and install the Amazon Redshift ODBC drivers on the Developer client and server machines.

Configuring System DSN

To use an ODBC connection to connect to Amazon Redshift, you must configure a System DSN in the ODBC datasource administrator based on the operating system.

Configuring a System DSN on Windows

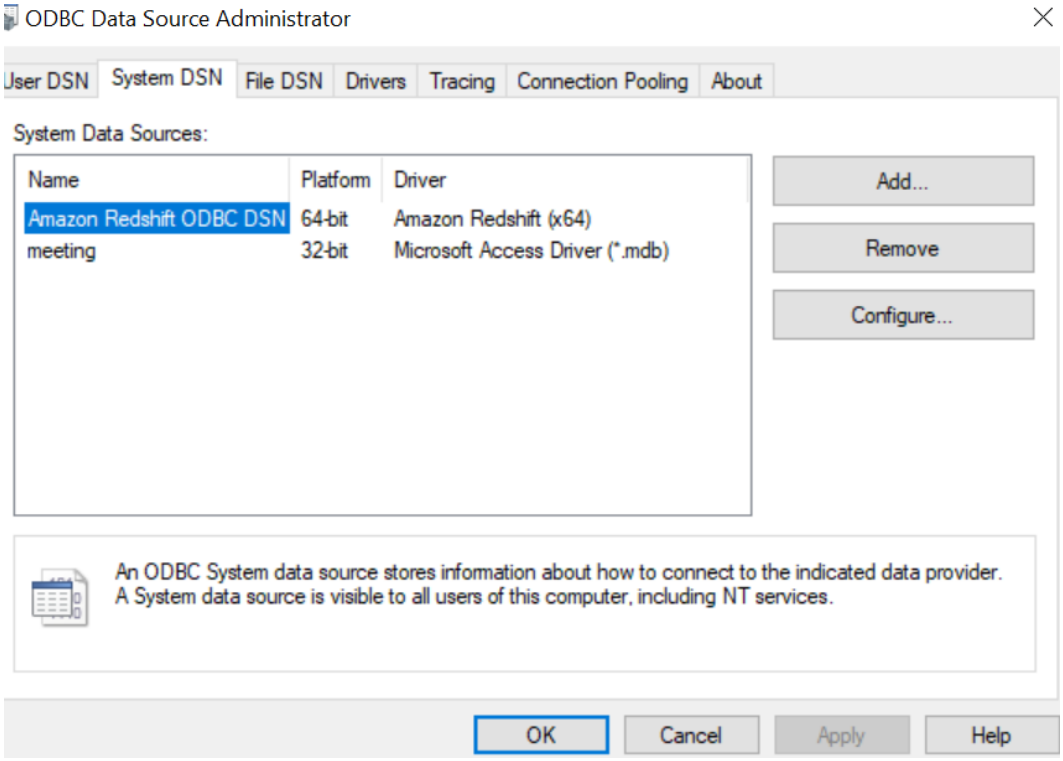
Configure a System DSN on Windows to use ODBC connection to connect to Amazon Redshift.

1. Open and double-click the `odbcad32.exe` ODBC data source file from the following location: .

   `C:\WINDOWS\system32`

   The **ODBC Data Sources Administrator** dialog box appears.
2. Click **System DSN**.

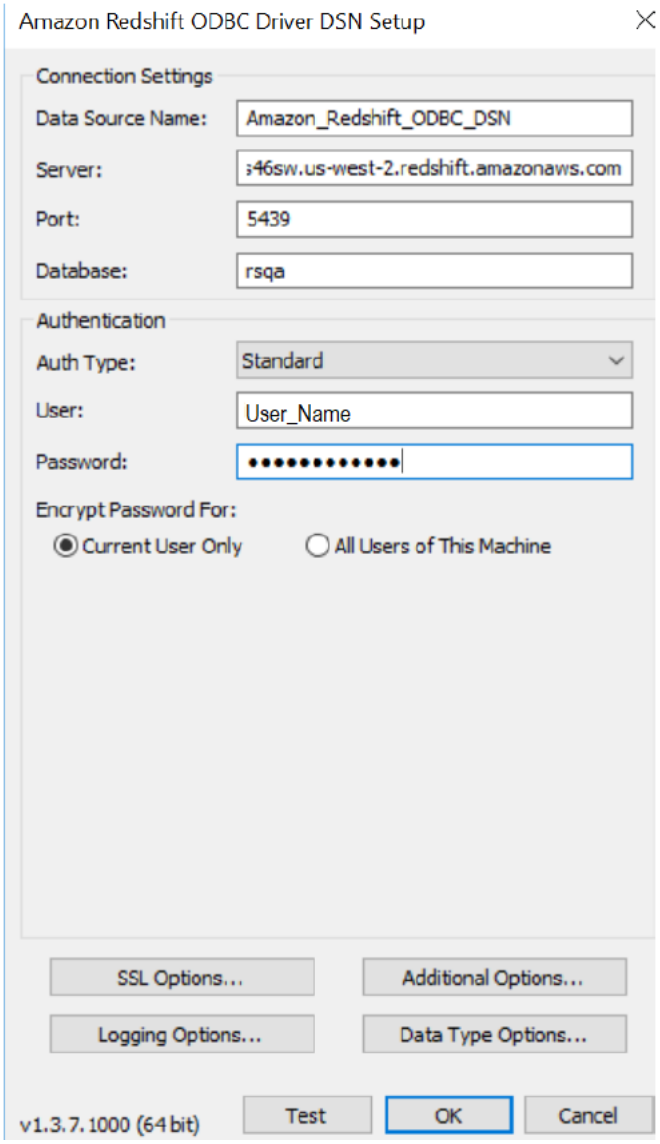
   The **System DSN** tab appears.
   The following image shows the **System DSN** tab on the **ODBC Data Sources Administrator** dialog box:



3. Click **Configure**.

   The **Amazon Redshift ODBC Driver DSN Setup** dialog box appears.

The following image shows the **Amazon Redshift ODBC Driver DSN Setup** dialog box where you can configure the connection settings and authentication:



4. Specify the following connection properties in the **Connection Settings** section:

Property	Description
Data Source Name	Name of the data source.
Server	Location of the Amazon Redshift server.
Port	Port number of the Amazon Redshift server.
Database	Name of the Amazon Redshift database.

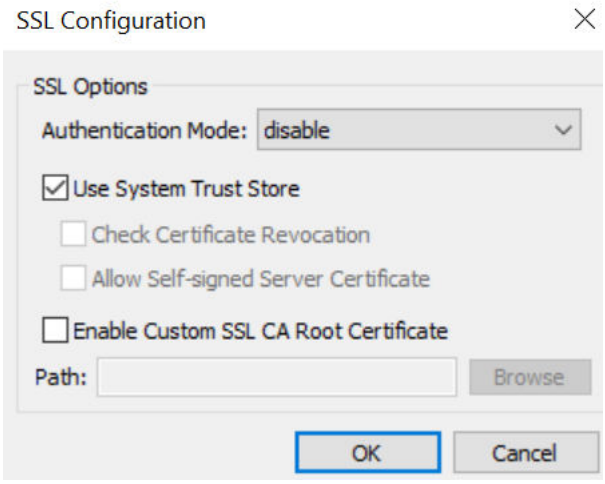
Note: You must specify the **Server**, **Port**, and **Database** values from the JDBC URL.

5. Specify the following authentication properties in the **Authentication** section:

Property	Description
Auth Type	Type of the authentication. Default is **Standard**.
User	User name to access the Amazon Redshift database.
Password	Password for the Amazon Redshift database.
Encrypt Password For	Encrypts the password for the following users: - **Current User Only** - **All Users of This Machine** Default is **Current User Only**.

6. Click **SSL Options** in the **Amazon Redshift ODBC Driver DSN Setup** dialog box.

   The **SSL Configuration** dialog box appears.
   The following image shows the **SSL Configuration** dialog box:



7. Select **disable** to disable the authentication in the **Authentication Mode** field.
8. Click **OK** in the **SSL Configuration** dialog box.

   The **SSL Configuration** dialog box closes.
9. Click **Test** to test the connection in the **Amazon Redshift ODBC Driver DSN Setup** dialog box.
10. Click **OK**.

Configuring a System DSN on Linux

Configure a System DSN on Linux to use ODBC connection to connect to Amazon Redshift.

1. Configure the `odbc.ini` file properties in the following format:

```
[ODBC Data Sources]
driver_name=dsn_name

[dsn_name]
```

```
Driver=path/driver_file

Host=cluster_endpoint
Port=port_number
Database=database_name
```

2. Specify the following properties in the `odbc.ini` file:

Property	Description
ODBC Data Sources	Name of the data source.
Driver	Location of the Amazon Redshift ODBC driver file.
Host	Location of the Amazon Redshift host.
Port	Port number of the Amazon Redshift server.
Database	Name of the Amazon Redshift database.

Note: You must specify the **Host**, **Port**, and **Database** values from the JDBC URL.

3. Add the `odbc.ini` file path in your source file in the following format:

```
ODBCINI=/<odbc.ini file path>/odbc.ini
```

4. Restart the Data Integration Service.

Creating an Amazon Redshift ODBC Connection

Create an Amazon Redshift ODBC connections to access Amazon Redshift read and write data objects.

1. In the Developer tool, click **New Connections**.

   The **Select a Connection Category** box appears.
2. Select **Connections**.
3. Specify the name, description, and location for the connection.
4. Select **Type** as **ODBC** and click **Next**.
5. Configure the following connection properties:

Relational Connection Property	Description
User Name	Enter the user name to connect to the Amazon Redshift database.
Password	Enter the password to connect to the Amazon Redshift database.
Connection String	Enter the name of the ODBC data source that you created for teh Amazon Redshift database on the Data Integration Service host machine. For example: `AmazonRSDW_ODBC_SERVER`

Relational Connection Property	Description
Code Page	Select the code page that the Data Integration Service uses to read or write data.
ODBC Provider	Select **AWS Redshift**.

6. Test the connection and click **OK**.

   The Amazon Redshift ODBC connection is created successfully.

Importing the Amazon Redshift Data Objects

Import the Amazon Redshift read data object from which you want to read data and the write data object to write data.

1. Select a project or folder in the **Object Explorer** view.
2. Click **File** > **New** > **Data Object**.
3. Select **Relational Data Object** and click **Next**.

   The **New Relational Data Object** dialog box appears.
4. Click **Browse** next to the **Connection** option and select the ODBC connection from which you want to import the Amazon Redshift resources.
5. Click **Create data from existing resource**.
6. To add a resource to the **Relational Data Object**, click **Browse** next to the **Resource** option.
7. Navigate to the resource to add it to the data object and click **Ok**.
8. Click **Browse** next to the **Location** option and select the target project or folder.
9. Click **Finish**.

The data object appears under **Data Object** in the project or folder in the **Object Explorer** view. You can also add resources to a relational data object after you create the data object.

Creating a Mapping

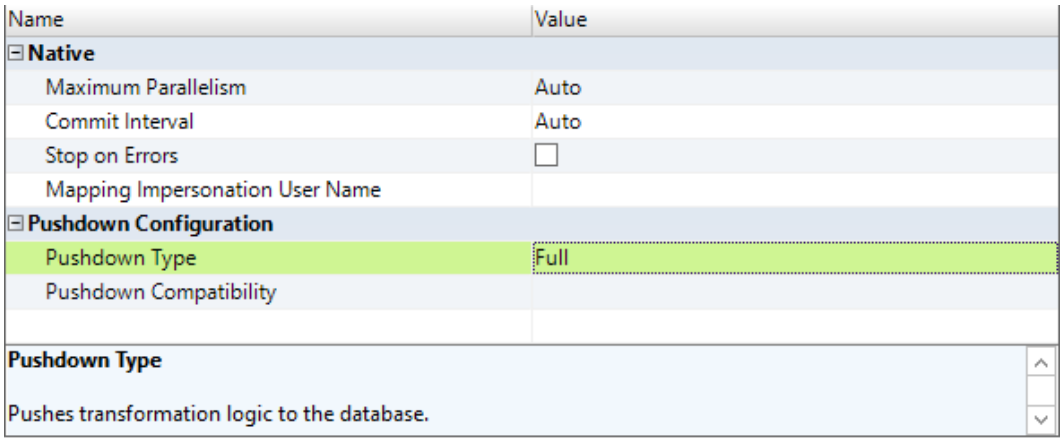
After you import the read and write data objects, create a mapping to read data from and write data to Amazon Redshift.

In a mapping, you define properties that determine how the Data Integration Service extracts data from or loads data to a data source. Configure pushdown optimization for the read and write data objects.

1. Select a project or folder in the **Object Explorer** view.
2. Click **Tasks** > **Create**.
3. Click **File** > **New** > **Mapping**.
4. Enter a mapping name and click **Finish**.

   An empty mapping appears in the editor.
5. Drag a data object to the editor and select **Read** to add the data object as a source.
6. Drag a data object to the editor and select **Write** to add the data object as a target.
7. On the **Properties** tab, select **Run-time**.
8. Select `Full` as pushdown type.

The following image shows the pushdown optimization configuration:



9. Save and run the mapping.

Supported Pushdown Optimization Functions and Operators

The following table summarizes the availability of pushdown functions in an Amazon Redshift database:

Function	Pushdown	Function	Pushdown	Function	Pushdown
ABORT()	No	INITCAP()	Yes	REG_MATCH()	No
ABS()	Yes	INSTR()	Yes	REG_REPLACE	No
ADD_TO_DATE()	Yes	IS_DATE()	No	REPLACECHR()	No
AES_DECRYPT()	No	IS_NUMBER()	No	REPLACESTR()	No
AES_ENCRYPT()	No	IS_SPACES()	No	REVERSE()	No
ASCII()	No	ISNULL()	Yes	ROUND(DATE)	No
AVG()	Yes	LAST()	No	ROUND(NUMBER)	Yes
CEIL()	Yes	LAST_DAY()	Yes	RPAD()	Yes
CHOOSE()	No	LEAST()	No	RTRIM()	Yes
CHR()	Yes	LENGTH()	Yes	SET_DATE_PART()	No
CHRCODE()	No	LN()	Yes	SIGN()	Yes
COMPRESS()	No	LOG()	No	SIN()	Yes
CONCAT()	Yes	LOOKUP	No	SINH()	No
COS()	Yes	LOWER()	Yes	SOUNDEX()	No
COSH()	No	LPAD()	Yes	SQRT()	Yes

Function	Pushdown	Function	Pushdown	Function	Pushdown
COUNT()	Yes	LTRIM()	Yes	STDDEV()	Yes
CRC32()	No	MAKE_DATE_TIME()	No	SUBSTR()	Yes
CUME()	No	MAX()	Yes	SUM()	Yes
DATE_COMPARE()	Yes	MD5()	No	SYSTIMESTAMP()	Yes
DATE_DIFF()	Yes	MEDIAN()	No	TAN()	Yes
DECODE()	Yes	METAPHONE()	No	TANH()	No
DECODE_BASE64()	No	MIN()	Yes	TO_BIGINT	Yes
DECOMPRESS()	No	MOD()	YesS	TO_CHAR(DATE)	Yes
ENCODE_BASE64()	No	MOVINGAVG()	No	TO_CHAR(NUMBER)	Yes
EXP()	Yes	MOVINGSUM()	No	TO_DATE()	Yes
FIRST()	No	NPER()	No	TO_DECIMAL()	Yes
FLOOR()	Yes	PERCENTILE()	No	TO_FLOAT()	Yes
FV()	No	PMT()	No	TO_INTEGER()	Yes
GET_DATE_PART()	Yes	POWER()	Yes	TRUNC(DATE)	Yes
GREATEST()	No	PV()	No	TRUNC(NUMBER)	Yes
IIF()	Yes	RAND()	No	UPPER()	Yes
IN()	Yes	RATE()	No	VARIANCE()	Yes
INDEXOF()	No	REG_EXTRACT()	No		

You can use the following operators lists to use in an Amazon Redshift database:

- + , - , *, /, %
- >, = , >= , <=
- AND, OR, NOT
- ||, !=, ^=

The following table lists the transformation types that PowerExchange for Amazon Redshift supports for the source-side or full pushdown optimization:

Transformation	Source-Side	Full
Aggregator	Yes	Yes
Expression	Yes	Yes

Transformation	Source-Side	Full
Filter	Yes	Yes
Joiner	Yes	Yes
Lookup	Yes	Yes
Router	No	Yes
Sorter	Yes	Yes **Note:** If you select the **Distinct rows** option in the Sorter transformation to push the transformation using full pushdown optimization, the Data Integration Service does not ignore the `ORDER BY` clause. If you select the **All rows** option in the Sorter transformation, the Data Integration Service ignores the `ORDER BY` clause.
Union	Yes	Yes
Update Strategy	No	Yes **Note:** To push the Update Strategy transformation using full pushdown optimization, do not select the **Forward Rejected Rows** option in the Update Strategy transformation. Additionally, in the **Run-time** tab in the mapping, select the value of the **Pushdown Compatibility** property as **Rows do not have the same key**.

Rules and Guidelines for Functions in Pushdown Optimization

Use the following rules and guidelines when pushing functions to an Amazon Redshift database:

- To use pushdown optimization, you must set the value of the **Multiple Matches** Lookup transformation property to **Return all rows**.
- To push TRUNC(DATE) to Amazon Redshift, you must define the date and format arguments. Otherwise, the Data Integration Service does not push the function to Amazon Redshift.
- To push TO_DATE() to Amazon Redshift, you must define the string and format arguments. Otherwise, the Data Integration Service does not push the function to Amazon Redshift.
- For Amazon Redshift, when you define only a string argument for TO_DATE() and TO_CHAR(), the Data Integration Service considers the default date format present in the session property. The default date format in the session property is: MM/DD/YYYY HH24:MI:SS.US
- Do not specify a format for SYSTIMESTAMP() to push the SYSTIMESTAMP to Amazon Redshift. The Amazon Redshift database returns the complete time stamp.
- The aggregator functions for Amazon Redshift accept only one argument, a field set for the aggregation function. The filter condition argument is not honored. In addition, make sure that all fields mapped to the target are listed in the GROUP BY clause.
- To push INSTR() to Amazon Redshift, you must only define string, search_value, and start arguments. Amazon Redshift does not support occurrence and comparison_type arguments.
- The flag argument is ignored when you push TO_BIGINT and TO_INTEGER to Amazon Redshift.

- The CaseFlag argument is ignored when you push IN() to Amazon Redshift.
- If you use the NS format as part of the ADD_TO_DATE() function, the Data Integration Service does not push the function to Amazon Redshift.
- If you use any of the following formats as part of the TO_CHAR() and TO_DATE() functions, the Data Integration Service does not push the function to Amazon Redshift:
  - NS
  - SSSS
  - SSSSS
  - RR
- To push TRUNC(DATE), GET_DATE_PART(), and DATE_DIFF() to Amazon Redshift, you must use the following formats:
  - D
  - HH24
  - MI
  - MM
  - MS
  - SS
  - US
  - YYYY
- To push GET_DATE_PART() to Amazon Redshift, you must use the following formats:
  - D
  - DDD
  - HH24
  - MI
  - MM
  - MS
  - SS
  - US
  - YYYY

Author

Subhashree Salam

Acknowledgements

The author would like to acknowledge Shivaprasad Yallappagoudar, for his technical assistance with this article.